

Algoritmi di classificazione ad albero

La prima parte della lezione inizia in maniera laboratoriale presentando una dataset simulato. E' stato chiesto agli studenti di provare "a ragionare" e fare una previsione della variabile Target

```
import pandas as pd
import numpy as np

np.random.seed(42)
n_rows = 50

# Creazione Variabili
eta = np.random.randint(1, 18, n_rows)
gravita = np.random.randint(1, 11, n_rows)
supporto = np.random.choice([0, 1], n_rows)

# Definizione Relazione
# Se il punteggio è positivo -> 1, altrimenti -> 0
punteggio = (0.6 * gravita) - (0.2 * eta) - (3.0 * supporto)
target = (punteggio > -1).astype(int)

# Creazione DataFrame
df_toy = pd.DataFrame({
    'Eta': eta,
    'Gravita': gravita,
    'Supporto_Familiare': supporto,
    'Target': target
})

print(df_toy.head())
print(f"\nBilanciamento Classi:\n{df_toy['Target'].value_counts()}")
```

	Eta	Gravita	Supporto_Familiare	Target
0	7	4	1	0
1	15	8	0	1
2	11	7	0	1
3	8	9	1	1
4	7	8	0	1

Bilanciamento Classi:

Target	Count
1	35
0	15

Name: count, dtype: int64

```
print(df_toy)
```

	Eta	Gravita	Supporto_Familiare	Target
0	7	4	1	0
1	15	8	0	1
2	11	7	0	1
3	8	9	1	1
4	7	8	0	1
5	11	5	0	1
6	11	2	0	0
7	4	5	0	1
8	8	8	0	1
9	3	10	1	1
10	2	9	1	1
11	12	9	1	1
12	6	1	0	1
13	2	9	0	1
14	1	7	0	1
15	12	9	1	1
16	12	8	0	1
17	17	1	0	0
18	10	8	1	1
19	16	8	0	1
20	15	3	1	0
21	15	1	1	0
22	12	8	1	1
23	3	3	0	1
24	5	3	0	1
25	7	1	0	1
26	9	5	0	1
27	7	10	0	1
28	4	7	0	1
29	14	10	0	1
30	9	9	1	1
31	2	7	0	1
32	15	9	1	1

```

33  7      8      0      1
34 12      2      0      0
35  8      1      0      0
36 15      7      1      0
37  3      7      1      1
38 14      8      1      0
39 17      5      1      0
40  4      3      0      1
41  8      8      1      1
42  4      6      0      1
43  2      3      0      1
44  6      1      1      0
45 10      3      1      0
46  4      5      1      1
47 12      3      1      0
48  2      1      1      0
49 10      5      1      0

```

```

# Qui solo prime righe del dataset su cui è stato chiesto di ragionare
# su come fare la previsione

```

```
df_toy.head(8)
```

	Eta	Gravita	Supporto_Familiare	Target
0	7	4	1	0
1	15	8	0	1
2	11	7	0	1
3	8	9	1	1
4	7	8	0	1
5	11	5	0	1
6	11	2	0	0
7	4	5	0	1

Esercitazione di gruppo:

Utilizzando i dati precedenti come potresti classificare ovvero prevedere la variabile target 0 e 1?

Varie ipotesi fatte in classe. In particolare

- Se la variabile "supporto familiare" è 0 ho 6 volte target=1 e 1 volta target=0
- A questo punto se la variabile gravità è < 5 separiamo e otteniamo delle foglie in cui abbiamo Target=0 una volta se gravità < 5 e Target=1 se ≥ 5 .
- Se la variabile "supporto familiare" è 1 ho 1 volta Target 1 e 1 volta target 0
- A questo punto usiamo la variabile gravità e in questo caso scegliamo il valore gravità < 9 . Con questa scelta abbiamo Target=0 che se gravità è minore di 9 e Target=1 viceversa

Proviamo con i successivi dati se funziona?

```

8  8  0  1
3 10  1  1
2  9  1  1

```

Pare che funzioni?

Come generalizziamo ?

Come potrebbe farlo un computer?

✓ Video utile per capire:

Analizziamo questo video riflettendo passo passo sui vari step che emergono.

https://www.youtube.com/watch?v=_L39rN6gz7Y

```
# qui stampa del dataset con più righe per vedere se idea funziona...
```

```
df_toy.head(15)
```

	Eta	Gravita	Supporto_Familiare	Target
0	7	4	1	0
1	15	8	0	1
2	11	7	0	1
3	8	9	1	1
4	7	8	0	1
5	11	5	0	1
6	11	2	0	0
7	4	5	0	1
8	8	8	0	1
9	3	10	1	1
10	2	9	1	1
11	12	9	1	1
12	6	1	0	1
13	2	9	0	1
14	1	7	0	1

Decision Tree - Algoritmi di Classificazione ad Albero

Dopo la prima parte laboratoriale proviamo a formalizzare le idee emerse si provare a separare le decisioni utilizzando delle soglie. L'idea dell'algoritmo è stata intuita ma è necessario formalizzarla.

Iniziamo a spiegare introducendo un po' più di rigore.

Gli algoritmi di classificazione ad albero (ma anche, vedremo, quelli di regressione) degli algoritmi fondamentali alla base del Machine Learning (ML).

Insieme alla regressione lineare e logistica, gli alberi decisionali possono essere considerati gli algoritmi base dell'apprendimento automatico.

Un **Decision Tree** (albero decisionale) è uno dei più intuitivi e interpretabili algoritmi di ML.

Intepretabili vuol dire che è possibile spiegare le motivazioni della classificazione ovvero quali sono le variabili che hanno contribuito a classificare in un modo o in un altro quei dati.

IDEA INTUITIVA: Funziona come un gioco a "Indovina il personaggio famoso": il gioco consiste nel porre una serie di domande a cui è possibile dare una risposta dicotomica (sì, no) e all'aumentare del numero di domande si arriva ad una una decisione finale.

Esempio classico:

Immagina di dover decidere se uscire con l'ombrello:

```
Devo portare l'ombrello?
```

```
|
|— Sta piovendo?
|   |— Sì → Porta l'ombrello
|   |— NO → Previsioni pioggia oggi?
|           |— Sì → Porta l'ombrello
|           |— NO → Non portarlo
|— (fine)
```

Questa struttura è esattamente un Decision Tree!

Caratteristiche Principali

Inseriamo una tabella che descrive le principali caratteristiche di un albero decisionale.

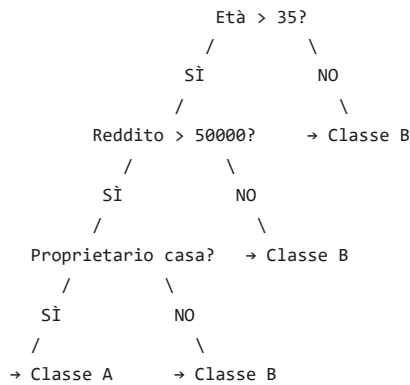
Caratteristica	Valore
Tipo	Algoritmo supervisionato
Task	Classificazione e Regressione
Interpretabilità	Alta (white-box)
Gestione variabile (feature) categoriche	Nativa

Caratteristica	Valore
Gestione valori mancanti	Dipende dall'implementazione
Scalabilità	Tendenza a overfitting
Velocità di training	Veloce

Come Funziona un Decision Tree

Il Decision Tree classifica un nuovo campione **attraversando l'albero dalla radice alle foglie**:

Campione nuovo: [età=45, reddito=60000, proprietario_casa=Sì]



Il campione segue il percorso: Età > 35 → Sì → Reddito > 50000 → Sì → Proprietario casa → Sì → **Classe A**

Il Processo di Training

Durante il training, l'algoritmo costruisce l'albero in modo **top-down** (dall'alto verso il basso):

STEP 1: Inizia con TUTTI i dati nel nodo radice
[tutti i campioni]

STEP 2: Trova la MIGLIORE feature e soglia per dividere
"Quale domanda separa meglio le classi?"

STEP 3: Dividi i dati in due sotto-insiemi
[campioni con feature ≤ soglia] | [campioni con feature > soglia]

STEP 4: Ripeti ricorsivamente per ogni sotto-albero
fino a un criterio di arresto

STEP 5: Le foglie contengono le predizioni finali

Esempio Visivo con Dati

Supponiamo di avere questi dati per classificare se un paziente è **a rischio (1)** o **non a rischio (0)**:

Paziente	Età	Pressione	Fumatore	Classe
P1	25	120	NO	0
P2	55	150	Sì	1
P3	30	125	NO	0
P4	60	160	Sì	1
P5	45	140	Sì	1
P6	35	130	NO	0
P7	50	145	NO	1
P8	28	118	NO	0

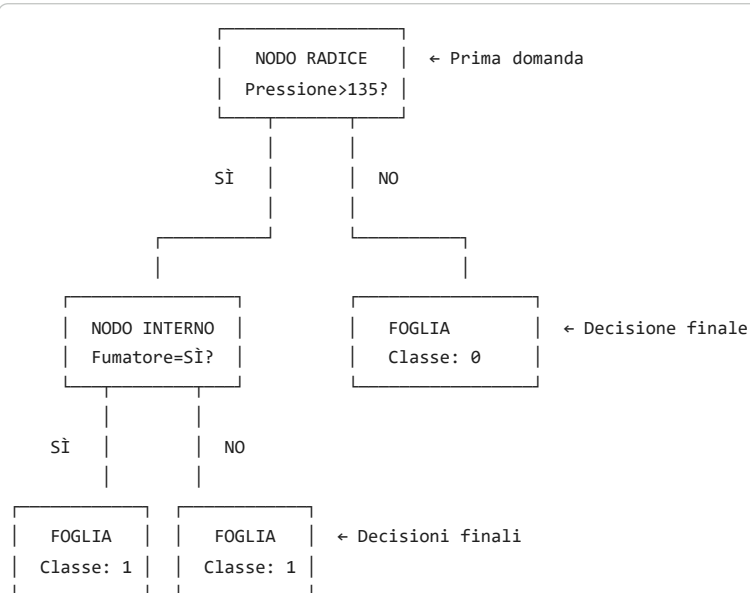
L'algoritmo cerca la migliore domanda per separare Classe 0 da Classe 1.

Possibili domande:

- "Età > 40?" → Separa abbastanza bene le classi
- "Pressione > 135?" → Separa ancora meglio
- "Fumatore = Sì?" → Buona separazione ma non perfetta

L'algoritmo sceglie la domanda che **minimizza l'impurezza** (vedremo come nel paragrafo successivo).

3. Struttura dell'Albero



Terminologia

Termine	Definizione
Nodo Radice	Il primo nodo, contiene tutti i dati
Nodo Interno	Un nodo che pone una domanda e ha figli
Foglia (Leaf)	Nodo finale che contiene la predizione
Profondità	Numero di livelli dell'albero
Branch	Il percorso tra due nodi
Split	La divisione dei dati in un nodo interno

Proprietà

- L'albero è sempre **binario** (ogni nodo ha esattamente 2 figli) in scikit-learn
- Le foglie sono **pure** idealmente (contengono una sola classe)
- La **profondità** determina la complessità del modello

Criteri di Split

Il centro di queste tipologie di algoritmi è dove separare le classi (split)

Ad ogni nodo l'algoritmo deve decidere:

1. **Quale variabile (feature)** usare per dividere?
2. **Quale soglia** usare?

La risposta: la combinazione che **massimizza la separazione** (o equivalentemente, **minimizza l'impurezza**) dei dati.

L'indice di GINI: Gini Impurity (Impurità/Impurezza di Gini)

Il criterio più usato (in scikit-learn) per la classificazione.

Formula:

$$\text{Gini} = 1 - \sum p_i^2$$

Dove p_i = proporzione della classe i nel nodo

Interpretazione:

- Gini = 0 → Nodo **puro** (una sola classe)
- Gini = 0.5 → Massima impurezza (classi perfettamente bilanciate, per classificazione binaria)

Esempio:

Nodo con 10 campioni: 7 classe 0, 3 classe 1

$$p_0 = 7/10 = 0.7$$

$$p_1 = 3/10 = 0.3$$

```
Gini = 1 - (0.72 + 0.32)
Gini = 1 - (0.49 + 0.09)
Gini = 1 - 0.58
Gini = 0.42
```

Gini per diversi scenari:

```
Tutti classe 0:    Gini = 1 - (12 + 02) = 0      → Puro ✓
Metà e metà:      Gini = 1 - (0.52 + 0.52) = 0.5 → Massima impurezza
70% classe 0:     Gini = 1 - (0.72 + 0.32) = 0.42
90% classe 0:     Gini = 1 - (0.92 + 0.12) = 0.19 → Abbastanza puro
```

Criterio dell'entropia: Entropy

La nozione di entropia deriva dalla fisica e può essere vista come il grado di disordine di un sistema. L'idea fisica è stata "generalizzata" da Shannon e adattata alla teoria dell'informazione.

Basata sulla teoria dell'informazione.

Formula:

$$\text{Entropy} = - \sum p_i \times \log_2(p_i)$$

Interpretazione:

- Entropy = 0 → Nodo puro
- Entropy = 1 → Massima impurezza (binario)

Vediamo l'esempio precedente:

```
Nodo: 7 classe 0, 3 classe 1

Entropy = -(0.7 × log2(0.7) + 0.3 × log2(0.3))
Entropy = -(0.7 × (-0.515) + 0.3 × (-1.737))
Entropy = -(-0.360 + (-0.521))
Entropy = 0.881
```

Information Gain

Per scegliere lo split migliore, calcoliamo l'**Information Gain**: la riduzione dell'impurezza dopo lo split.

Formula:

$$\text{Information Gain} = \text{Impurezza}(\text{padre}) - \sum (n_i/N) \times \text{Impurezza}(\text{figlio}_i)$$

Dove:

n_i = numero di campioni nel figlio i
 N = numero totale di campioni nel padre

Esempio completo:

```
Padre: [7 classe 0, 3 classe 1] → Gini_padre = 0.42

Split su "Età > 40":
  Figlio sinistro (Età ≤ 40): [6 classe 0, 1 classe 1] → n=7
  Figlio destro (Età > 40):   [1 classe 0, 2 classe 1] → n=3

Gini_sinistro = 1 - (6/7)2 - (1/7)2 = 1 - 0.735 - 0.020 = 0.245
Gini_destro   = 1 - (1/3)2 - (2/3)2 = 1 - 0.111 - 0.444 = 0.445

Gini_dopo = (7/10) × 0.245 + (3/10) × 0.445
          = 0.1715 + 0.1335
          = 0.305

Information Gain = 0.42 - 0.305 = 0.115
```

L'algoritmo confronta l'Information Gain di **tutti** gli split possibili e sceglie quello con il valore più alto.

Confronto Gini vs Entropy

Caratteristica	Gini	Entropy
Range	[0, 0.5]	[0, 1]
Velocità di calcolo	Più veloce	⚠️ Più lento (logaritmo)
Comportamento	Tende a scegliere split più puri	Più sensibile alle minoranze
Default scikit-learn	Sì	No
Differenza pratica	Spesso risultati molto simili	

Inizia a programmare o [genera](#) codice con l'IA.

Fai doppio clic (o premi Invio) per modificare